

Software Requirements Specification and Design

Version 2.0

February 25, 2026

INTELLIGENT SCHEDULE MANAGER

Donyi Seng Kumbang

Aayush Jaiswal

Mohd. Saif Mansoori

Yash Kumawat

Arpit Suman

Semester IV



Department of Computer Science and Engineering
Faculty of Technology, University of Delhi

Submitted to:

Dr. Rekha R

Assistant Professor

Department of Computer Science and Engineering

Prepared for Academic & Development Use

Based on IEEE Std 830–1984 (SRS)

Revision History

Version	Date	Author	Description
1.0	27/01/2026	Project Team	Initial Version
2.0	22/02/2026	Project Team	Systematic Version

Approval Page

Name / Role	Date	Signature

Contents

Revision History	1
1 Introduction	6
1.1 Purpose	6
1.2 Scope	6
1.3 Acronyms, and Abbreviations	6
1.4 References	7
1.5 Overview	7
2 Overall Description	7
2.1 Product Perspective	7
2.2 Product Functions	7
2.3 User Characteristics	8
2.4 Operating Environment	8
2.5 Constraints	9
2.6 Assumptions and Dependencies	9
2.7 User Use Cases	9
2.7.1 User Use Case Diagram	10
2.7.2 Use Case: User Registration and Authentication	10
2.7.3 Use Case: Task Management	11
2.7.4 Use Case: Generate Schedule	11
2.7.5 Use Case: Habit Tracking	12
2.7.6 Use Case: Notification and Reminder	12
2.7.7 Use Case: Calendar Synchronization	13
2.7.8 Use Case: Logout	13
2.8 Common Dashboard Use Cases	14
2.8.1 Common Dashboard Use Case Diagram	14
2.8.2 Use Case: Display Dashboard and Schedule Overview	14
2.8.3 Use Case: Help / Contact Support	15
2.8.4 Use Case: About Application	16
2.8.5 Use Case: View Terms and Conditions	16
2.8.6 Use Case: Logout	16
3 Specific Requirements	17
3.1 Functional Requirements	17
3.2 Performance Requirements	18
3.3 Design Constraints	18

3.4	External Interface Requirements	18
3.4.1	User Interface	18
3.4.2	Software Interfaces	18
3.5	Non-Functional Requirements	18
4	Software Design	19
4.1	System Architecture	19
4.2	Detailed Design	21
4.2.1	Class Diagrams	21
4.3	Sequence Diagrams	22
4.3.1	Sequence Diagram – User Login	22
4.3.2	Sequence Diagram – Generate Schedule	23
4.3.3	Sequence Diagram – Reschedule After Task Update	23
4.3.4	Activity Diagram – Daily Task Lifecycle	24
4.3.5	State Diagram – Task State Model	24
4.3.6	Component Diagram – Intelligent Schedule Manager	25
4.3.7	Deployment Diagram – Intelligent Schedule Manager System	26
4.4	Database Schema	26
4.5	Security Design	29
4.5.1	Authentication and Authorization Mechanisms	29
4.5.2	Security Protocols	29
5	Traceability	29
5.1	Requirement-to-Design Mapping	29
A	Technology Stack	30
B	Development Tools	30
C	Database Notes	31
D	Future Enhancements	31

List of Figures

1	User use case diagram	10
2	Use Case: User Registration and Authentication	10
3	Use Case: Task Management	11
4	Use Case: Generate Schedule	11
5	Use Case: Habit Tracking	12
6	Use Case: Notification and Reminder	12
7	Use Case: Calendar Synchronization	13
8	Use Case: Logout	13
9	Common Dashboard Use Case Diagram	14
10	Use Case: Display Dashboard and Schedule Overview	15
11	Use Case: Help / Contact Support	15
12	Use Case: About Application	16
13	Use Case: View Terms and Conditions	16
14	Use Case: Logout	17
15	System Architecture Flowchart of Intelligent Schedule Manager	20
16	Class Diagram	21
17	Sequence Diagram	22
18	Sequence Diagram – User Login	22
19	Sequence Diagram – Generate Schedule	23
20	Sequence Diagram – Reschedule After Task Update	24
21	Activity Diagram – Daily Task Lifecycle	24
22	State Diagram – Task State Model	25
23	Component Diagram – Intelligent Schedule Manager	25
24	Deployment Diagram – Intelligent Schedule Manager System	26
25	Entity Relationship Diagram for Intelligent Schedule Manager	28

1 Introduction

This Software Requirements Specification (SRS) document provides a structured overview of the Intelligent Schedule Manager system. It defines the functional and non-functional requirements, system constraints, and design assumptions that guide the development lifecycle.

1.1 Purpose

The purpose of this document is to formally specify what the Intelligent Schedule Manager system should do and how it should behave under various conditions. It serves as a single source of truth for stakeholders, developers, and evaluators.

1.2 Scope

The Intelligent Schedule Manager is a mobile-first productivity application that assists users in managing time efficiently through AI-powered schedule generation, habit tracking, calendar synchronization, and minimal-distraction interfaces.

1.3 Acronyms, and Abbreviations

Acronym	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
SRS	Software Requirements Specification
UI	User Interface
ML	Machine Learning
HTTPS	HyperText Transfer Protocol Secure
TLS	Transport Layer Security
DB	Database
ER	Entity Relationship
EER	Enhanced Entity Relationship
CRUD	Create, Read, Update, Delete
OS	Operating System
IDE	Integrated Development Environment
UML	Unified Modeling Language
FR	Functional Requirement
NFR	Non-Functional Requirement
SQL	Structured Query Language
JSON	JavaScript Object Notation

1.4 References

- IEEE Std 830–1984, IEEE Guide to Software Requirements Specifications
- Karl Wieggers, Software Requirements
- Google Calendar API Documentation

1.5 Overview

This document is organized into major sections describing system context, requirements, interfaces, diagrams, and future enhancements.

2 Overall Description

This section provides a high-level overview of the Intelligent Schedule Manager system. It describes the system's position within a broader context, its major functions, intended users, operating environment, constraints, and assumptions. The purpose of this section is to help readers understand how the system fits into its environment before examining detailed functional requirements.

2.1 Product Perspective

The Intelligent Schedule Manager is a standalone, mobile-first productivity application designed to assist users in managing tasks, schedules, and habits in an intelligent and adaptive manner. The system is not an extension of any existing product; instead, it operates as an independent software solution while integrating with external services such as calendar platforms and notification providers.

From an architectural perspective, the system follows a modular client–server model. The mobile frontend serves as the primary user interface and interacts with cloud-based backend services through secure APIs. The backend is responsible for data management, business logic, and coordination with the AI scheduling engine. The AI engine performs computationally intensive tasks such as schedule optimization and dynamic rescheduling. External services, including calendar APIs and notification services, are integrated to enhance functionality and user experience.

This separation of concerns ensures scalability, maintainability, and flexibility, allowing individual components to evolve independently without affecting the overall system stability.

2.2 Product Functions

The Intelligent Schedule Manager provides a comprehensive set of functions aimed at improving user productivity and reducing cognitive load. The major functions of the system include,

but are not limited to, the following:

- User registration, authentication, and secure session management
- Creation, modification, and deletion of tasks with priorities and deadlines
- AI-based generation of optimized daily and weekly schedules
- Dynamic rescheduling of tasks in response to delays or early completion
- Habit tracking with progress visualization and analytics
- Synchronization of schedules with external calendar services
- Delivery of notifications and reminders for tasks and habits
- Optional reward-based motivation mechanisms such as streaks and achievements

Each of these functions works together to provide a cohesive productivity system that not only records user tasks but actively assists users in deciding how and when tasks should be performed.

2.3 User Characteristics

The primary users of the Intelligent Schedule Manager are regular smartphone users who wish to manage their time more effectively. These users may include students, working professionals, and individuals seeking better personal organization.

Users are assumed to have basic familiarity with mobile applications, such as navigating menus, entering text, and interacting with notifications. No advanced technical expertise is required to use the system. The system is designed with simplicity and usability in mind to ensure accessibility for users with varying levels of digital literacy.

The system does not distinguish between multiple user roles; all users interact with the same set of core features. However, personalization is achieved through user-specific preferences, habits, and historical usage data.

2.4 Operating Environment

The Intelligent Schedule Manager is designed to operate primarily on Android-based mobile devices, with potential cross-platform support through frameworks such as Flutter. The front-end application runs on user devices, while backend services are hosted in a cloud computing environment.

The system requires persistent internet connectivity for features such as user authentication, schedule synchronization, AI-based processing, and external calendar integration. Local device

resources such as storage and memory are utilized minimally, as most data processing and storage are handled on the backend.

The backend infrastructure is expected to run on scalable cloud servers capable of handling concurrent user requests and computational workloads generated by the AI scheduling engine.

2.5 Constraints

The design and operation of the Intelligent Schedule Manager are subject to several constraints. The system depends on third-party services such as calendar APIs and notification platforms, which may impose usage limits, latency, or policy changes.

Mobile device constraints, including limited processing power, battery life, and network availability, may also affect system performance. Additionally, the system must comply with platform-specific guidelines and security requirements imposed by mobile operating systems and application marketplaces.

These constraints have been carefully considered in the system design to ensure robust and reliable operation under real-world conditions.

2.6 Assumptions and Dependencies

The development and operation of the Intelligent Schedule Manager are based on certain assumptions. It is assumed that users possess compatible mobile devices and have access to a stable internet connection. Users are also expected to provide accurate task information and preferences for effective schedule generation.

The system depends on the continued availability and stability of external services, including calendar APIs and authentication providers. Any significant changes in these services may require updates to the system.

Furthermore, it is assumed that cloud infrastructure and AI processing services will remain scalable and available to support the anticipated number of users and workload.

2.7 User Use Cases

This subsection describes the major use cases associated with the Intelligent Schedule Manager. These use cases define how users interact with the system to manage tasks, generate schedules, track habits, and maintain productivity. The system is designed for regular users, including students, professionals, and individuals seeking efficient time management.

2.7.1 User Use Case Diagram

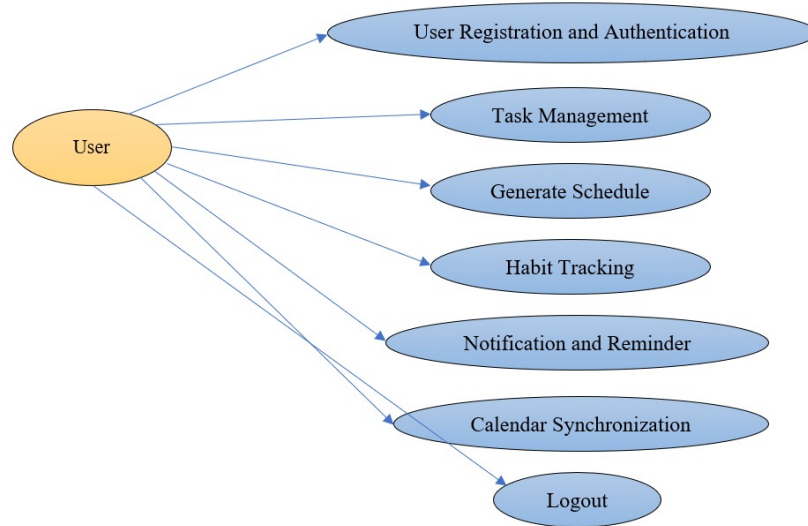


Figure 1: User use case diagram

The User Use Case Diagram illustrates the interaction between the user and the Intelligent Schedule Manager system. It shows how users perform essential operations such as registering, creating tasks, generating schedules, and receiving reminders. The diagram provides a high-level overview of system functionality from the user's perspective.

2.7.2 Use Case: User Registration and Authentication

Description: This use case allows new users to create an account and existing users to securely log in.

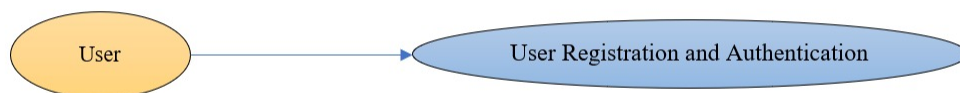


Figure 2: Use Case: User Registration and Authentication

Actors: User

Preconditions: User must have a valid email address and internet connectivity.

Main Flow:

1. User opens the application.

2. User selects Register or Login.
3. User enters credentials.
4. System validates credentials.
5. System grants access to dashboard.

Postconditions: User account is created or authenticated successfully.

2.7.3 Use Case: Task Management

Description: This use case allows users to create, update, and delete tasks.

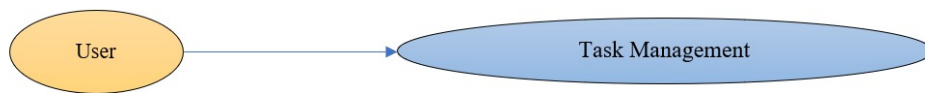


Figure 3: Use Case: Task Management

Actors: User

Main Flow:

1. User selects Create Task.
2. User enters task details.
3. System saves task to database.
4. User can modify or delete tasks later.

Postconditions: Task is stored and available for scheduling.

2.7.4 Use Case: Generate Schedule

Description: The system generates an optimized schedule using AI based on tasks and preferences.

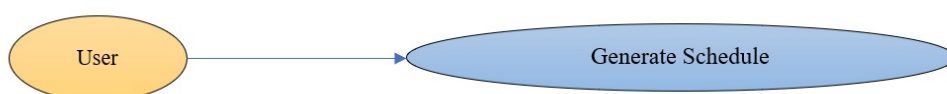


Figure 4: Use Case: Generate Schedule

Actors: User, AI Engine

Main Flow:

1. User selects Generate Schedule.
2. System retrieves tasks and preferences.
3. AI engine creates optimized schedule.
4. Schedule is stored and displayed.

Postconditions: Optimized schedule is available.

2.7.5 Use Case: Habit Tracking

Description: Users can track recurring habits and monitor their progress.

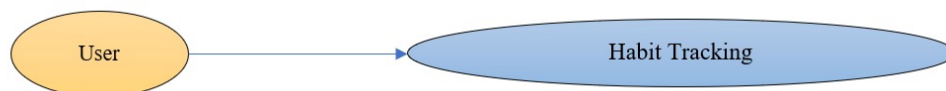


Figure 5: Use Case: Habit Tracking

Actors: User

Main Flow:

1. User creates habit.
2. User marks habit completion.
3. System updates streak and statistics.

Postconditions: Habit progress is updated.

2.7.6 Use Case: Notification and Reminder

Description: System sends reminders for tasks and habits.

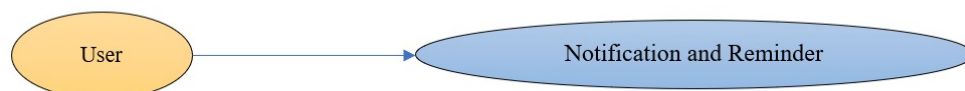


Figure 6: Use Case: Notification and Reminder

Actors: User, Notification Service

Main Flow:

1. System schedules reminder.
2. Notification service triggers alert.
3. User receives reminder.

Postconditions: User is notified of upcoming tasks.

2.7.7 Use Case: Calendar Synchronization

Description: Synchronizes schedule with external calendar services.

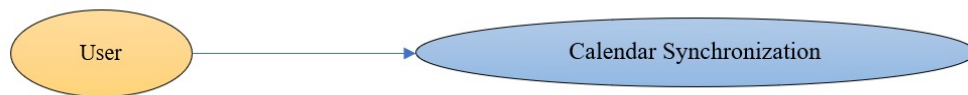


Figure 7: Use Case: Calendar Synchronization

Actors: User, Calendar API

Main Flow:

1. User enables calendar sync.
2. System connects to calendar API.
3. Schedule is exported to calendar.

Postconditions: Calendar is updated with schedule.

2.7.8 Use Case: Logout

Description: User securely logs out of the system.

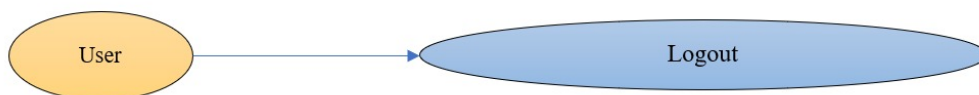


Figure 8: Use Case: Logout

Actors: User

Main Flow:

1. User selects logout.
2. System clears session.
3. User returns to login screen.

Postconditions: Session is terminated securely.

2.8 Common Dashboard Use Cases

The Common Dashboard represents the primary interface that users interact with after successful authentication. It provides access to core features such as viewing schedules, managing tasks and habits, accessing help resources, and logging out. These use cases describe system functionality that is available to all authenticated users.

2.8.1 Common Dashboard Use Case Diagram

The Common Dashboard Use Case Diagram illustrates the interaction between the user and the dashboard interface. It highlights actions such as viewing schedules, accessing help, re-viewing application information, and logging out. This diagram provides a visual overview of dashboard-level system interactions.

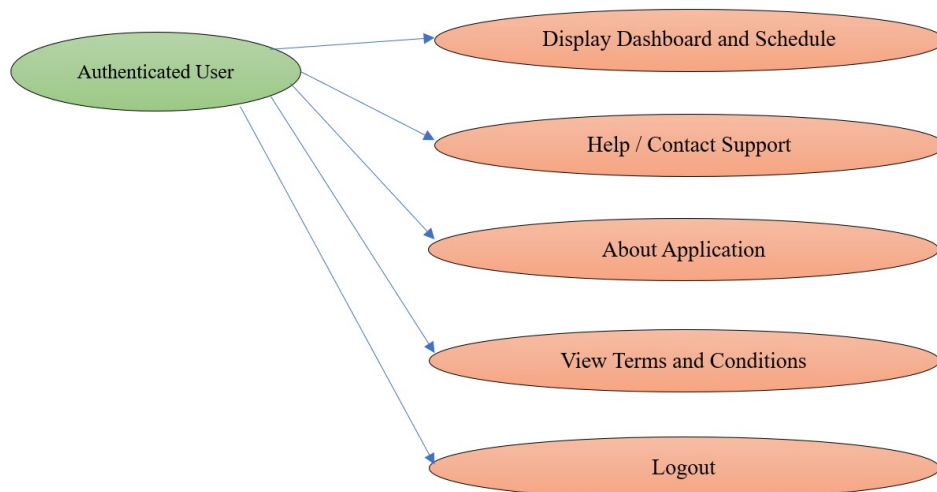


Figure 9: Common Dashboard Use Case Diagram

2.8.2 Use Case: Display Dashboard and Schedule Overview

Description: Allows the user to view their daily schedule, upcoming tasks, and habit progress.

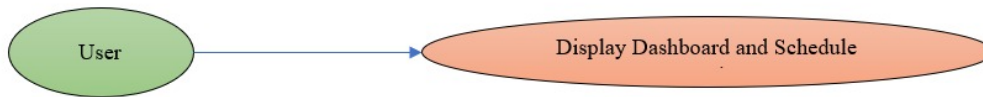


Figure 10: Use Case: Display Dashboard and Schedule Overview

Actors: User

Preconditions: User must be logged in.

Main Flow:

1. User logs into the system.
2. Dashboard loads automatically.
3. System retrieves schedule and tasks.
4. Schedule and productivity summary are displayed.

Postconditions: User can view and manage tasks and schedules.

2.8.3 Use Case: Help / Contact Support

Description: Allows users to access help resources or contact support for assistance.

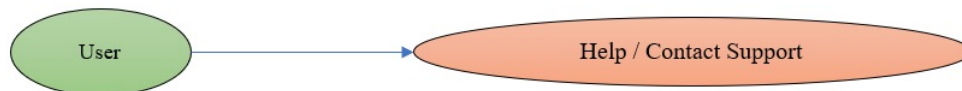


Figure 11: Use Case: Help / Contact Support

Actors: User

Main Flow:

1. User selects Help or Support option.
2. System displays help documentation.
3. User may submit a support request.

Postconditions: User receives assistance information.

2.8.4 Use Case: About Application

Description: Displays information about the Intelligent Schedule Manager application.

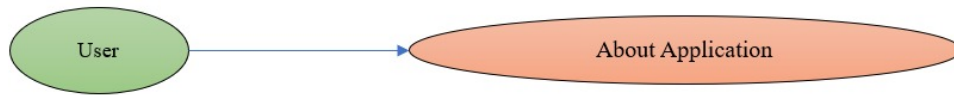


Figure 12: Use Case: About Application

Actors: User

Main Flow:

1. User selects About option.
2. System displays application version, developer details, and description.

Postconditions: User can view application information.

2.8.5 Use Case: View Terms and Conditions

Description: Allows users to view application policies and usage terms.

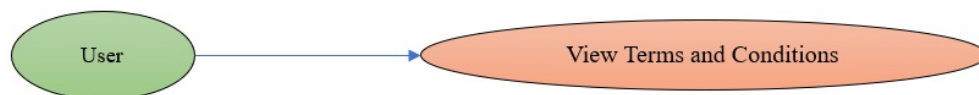


Figure 13: Use Case: View Terms and Conditions

Actors: User

Main Flow:

1. User selects Terms and Conditions.
2. System displays policy information.

Postconditions: User understands application policies.

2.8.6 Use Case: Logout

Description: Allows users to securely terminate their session.

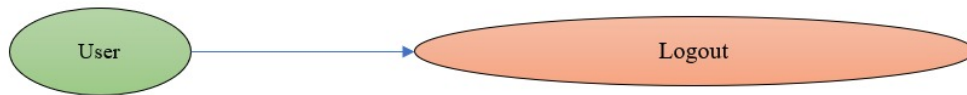


Figure 14: Use Case: Logout

Actors: User

Main Flow:

1. User selects Logout.
2. System clears authentication token.
3. User is redirected to login screen.

Postconditions: User session is terminated securely.

3 Specific Requirements

3.1 Functional Requirements

This section describes the core functional capabilities of the Intelligent Schedule Manager. Each functional requirement defines a specific behavior that the system must exhibit in order to fulfill user needs and achieve the intended objectives of intelligent scheduling, productivity enhancement, and user-centric task management.

FR-1: Secure User Registration and Login

The system shall provide functionality for secure user registration and authentication. Users shall be able to create an account using valid credentials such as email and password. During login, the system shall verify user credentials through the authentication service and grant access only upon successful validation. Security mechanisms such as encrypted password storage and token-based session management shall be employed to prevent unauthorized access.

FR-2: Task Creation, Update, and Deletion

The system shall allow authenticated users to create, modify, and delete tasks. Each task shall include attributes such as title, description, priority, deadline, and estimated duration. Users shall be able to update task details at any time, and delete tasks that are no longer required. Any modification to tasks shall be reflected immediately in the system and considered during schedule generation or rescheduling.

FR-3: AI-Based Schedule Generation

The system shall generate optimized daily and weekly schedules using an AI-based scheduling engine. The schedule generation process shall consider user-defined tasks, priorities, deadlines, habits, and preferences. The AI engine shall compute an optimal task sequence that

maximizes productivity while minimizing cognitive overload. The generated schedule shall be stored and presented to the user in an easy-to-understand format.

FR-4: Dynamic Task Rescheduling

The system shall support dynamic rescheduling of tasks when changes occur. If a task is delayed, completed earlier, or modified by the user, the system shall automatically recompute the remaining schedule. The AI engine shall adjust task ordering and time allocation to ensure deadlines are respected and user workload remains balanced.

FR-5: External Calendar Synchronization

The system shall provide synchronization with external calendar services such as Google Calendar. Upon user authorization, scheduled tasks and reminders shall be exported to the external calendar. Any changes in the internal schedule shall be updated accordingly to maintain consistency between the Intelligent Schedule Manager and the external calendar platform.

FR-6: Habit Tracking and Analytics

The system shall enable users to track recurring habits and monitor their progress over time. For each habit, the system shall maintain data such as completion history, streak count, and frequency. Analytical insights including visual charts, statistics, and trends shall be displayed to help users evaluate their consistency and long-term productivity improvements.

3.2 Performance Requirements

Schedule generation shall complete within 2 seconds under normal load.

3.3 Design Constraints

Implementation shall use Flutter or native Android technologies.

3.4 External Interface Requirements

3.4.1 User Interface

The UI shall be minimalistic, distraction-free, and intuitive.

3.4.2 Software Interfaces

The system shall interface with Google Calendar API, authentication services, and notification services.

3.5 Non-Functional Requirements

- Usability: Minimal learning curve
- Reliability: 99% uptime

- Security: Encrypted data at rest and in transit
- Scalability: Support growing user base
- Portability: Cross-platform support

4 Software Design

4.1 System Architecture

The Intelligent Schedule Manager follows a modular, client–server architecture designed to ensure scalability, maintainability, and extensibility. The system is divided into four primary architectural layers: Presentation Layer (Mobile Frontend), Application Layer (Backend API), AI Processing Layer, and Data Layer (Database).

The Mobile Application communicates with the Backend API via secure HTTPS protocols. The backend handles business logic, authentication, task processing, and communication with the AI scheduling engine. The AI engine performs optimization algorithms for schedule generation and rescheduling. The Database layer ensures persistent storage of user data, tasks, schedules, and habits.

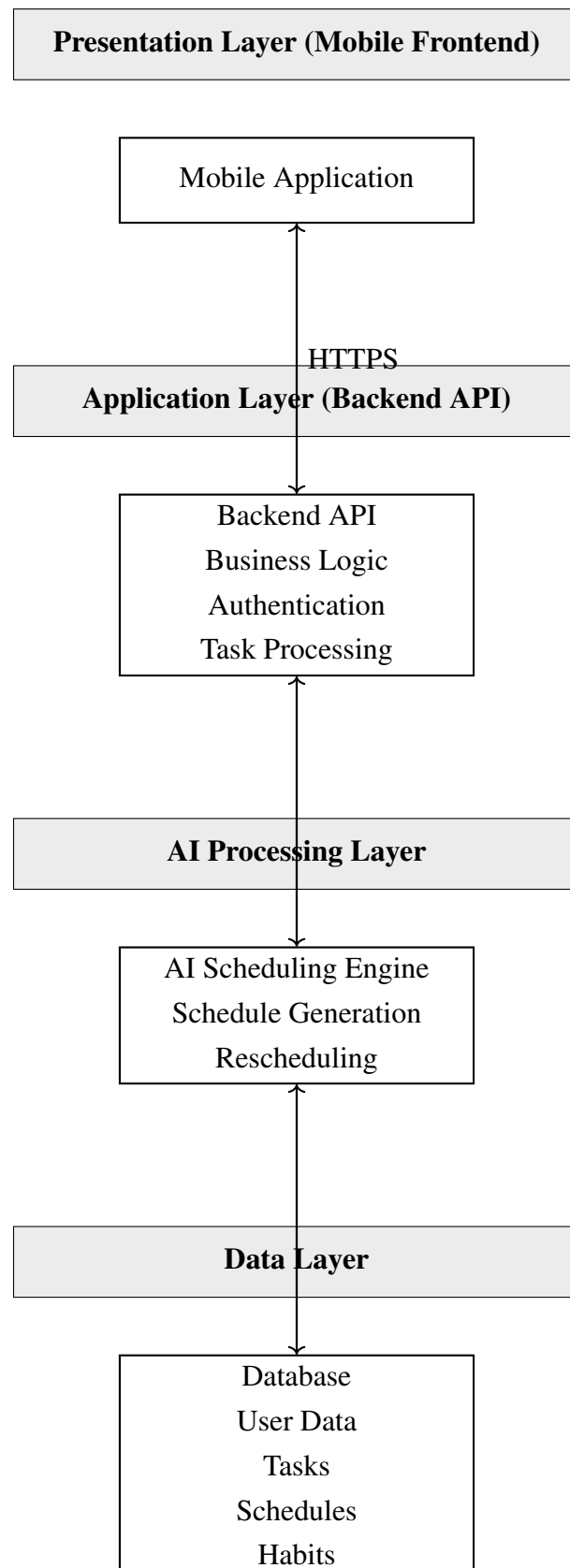


Figure 15: System Architecture Flowchart of Intelligent Schedule Manager

4.2 Detailed Design

4.2.1 Class Diagrams

The system class structure is designed using object-oriented principles. Major classes include User, Task, Habit, Schedule, and Notification. Each class encapsulates relevant attributes and behaviors. Relationships between classes ensure proper association, aggregation, and inheritance where applicable.

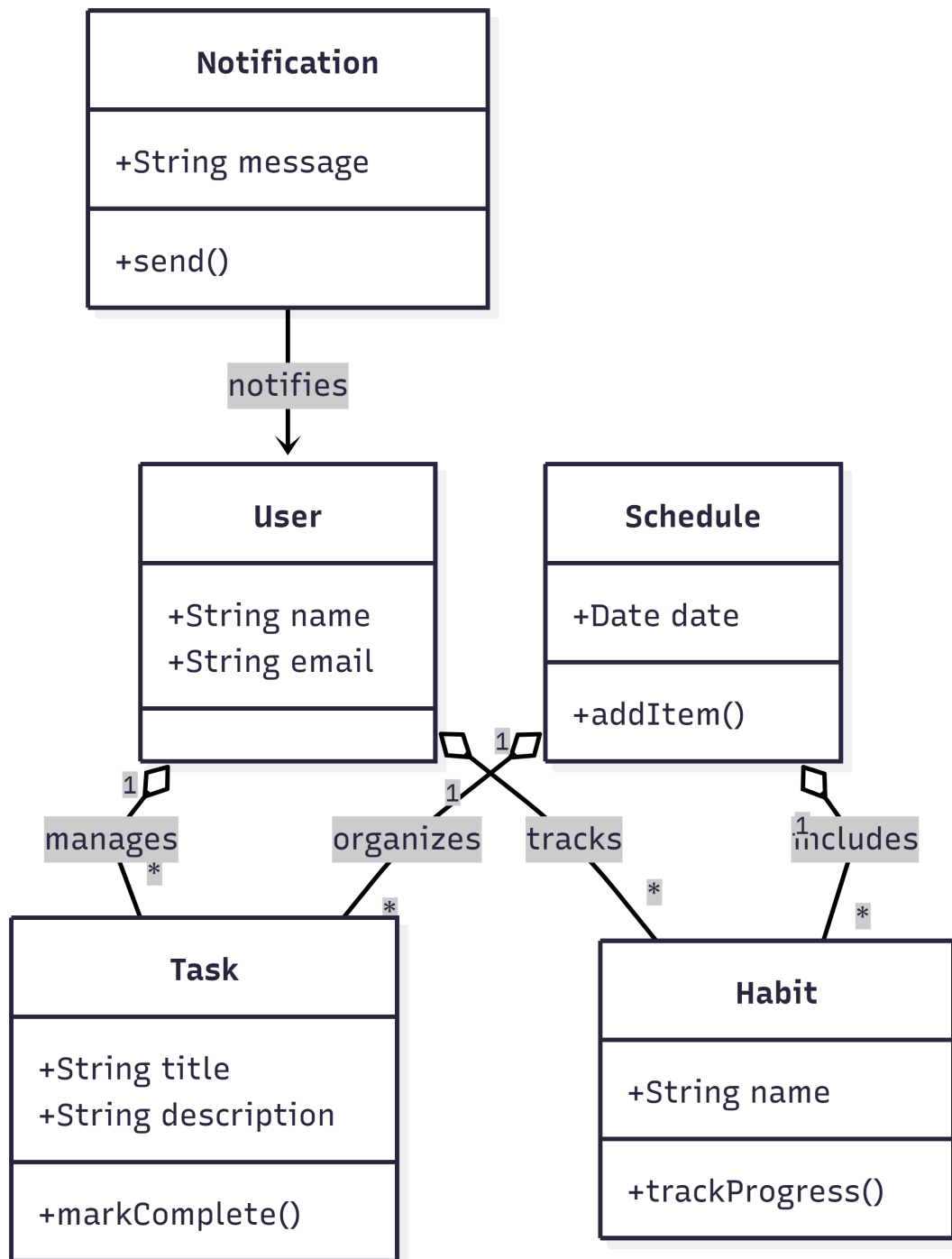


Figure 16: Class Diagram

4.3 Sequence Diagrams

This section presents the graphical models used to describe the behavior, structure, and deployment of the Intelligent Schedule Manager system. These diagrams help in visualizing system interactions, control flow, data organization, and architectural decisions, thereby improving understanding of system functionality and design.

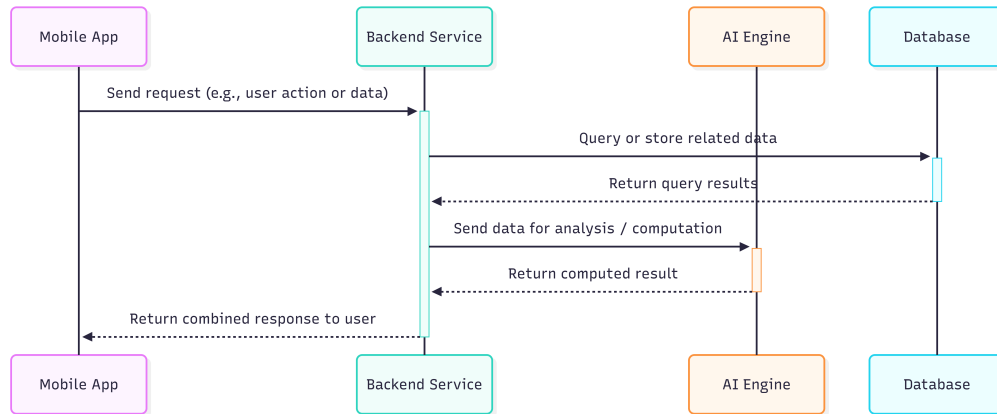


Figure 17: Sequence Diagram

4.3.1 Sequence Diagram – User Login

The sequence diagram illustrates the authentication workflow of the system. It shows how user credentials are entered through the mobile application and validated by backend and authentication services. The interaction with the database ensures secure verification of user data. Upon successful authentication, an access token is returned and the user is granted access to the dashboard.

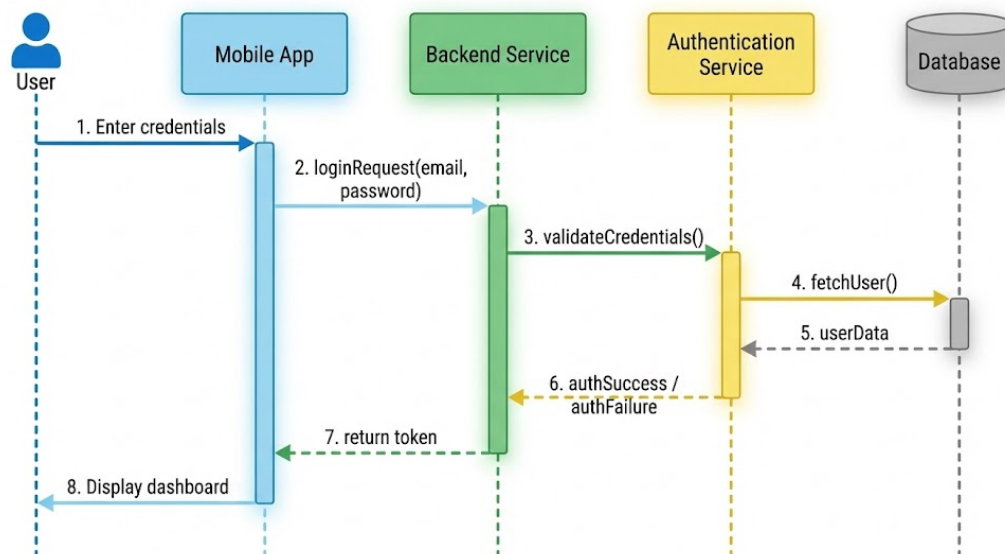


Figure 18: Sequence Diagram – User Login

4.3.2 Sequence Diagram – Generate Schedule

This diagram represents the interaction flow for generating an optimized daily schedule. It demonstrates how the backend retrieves tasks and user preferences from the database and invokes the AI scheduling engine. The optimized schedule is stored and reminders are scheduled before being returned to the user. This diagram highlights the AI-driven decision-making process of the system.

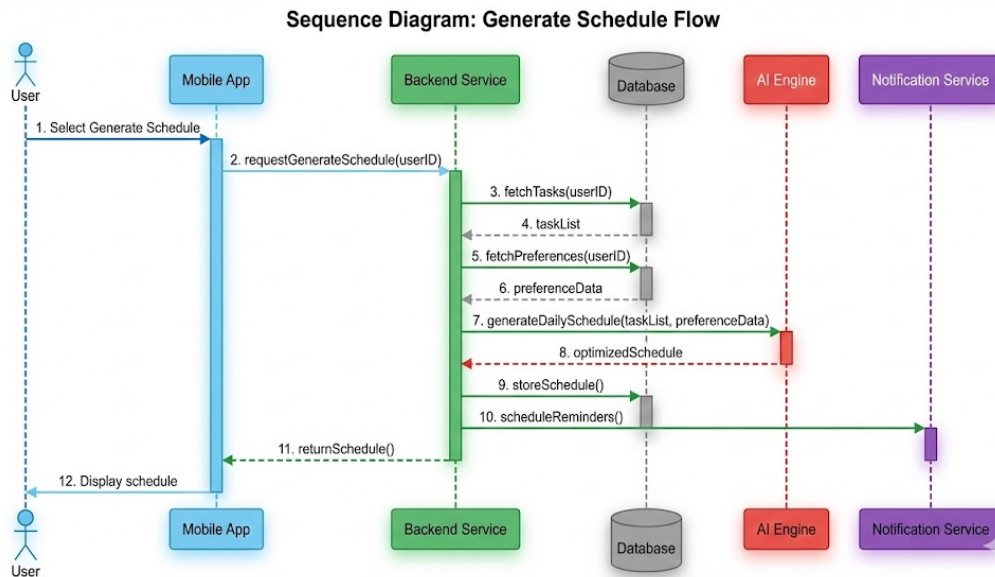


Figure 19: Sequence Diagram – Generate Schedule

4.3.3 Sequence Diagram – Reschedule After Task Update

This sequence diagram shows how the system responds when a task is updated or delayed. The backend service updates task information in the database and invokes the AI engine to recompute the schedule. The revised schedule is stored and returned to the user. This diagram emphasizes the system's capability to dynamically adapt to task changes.

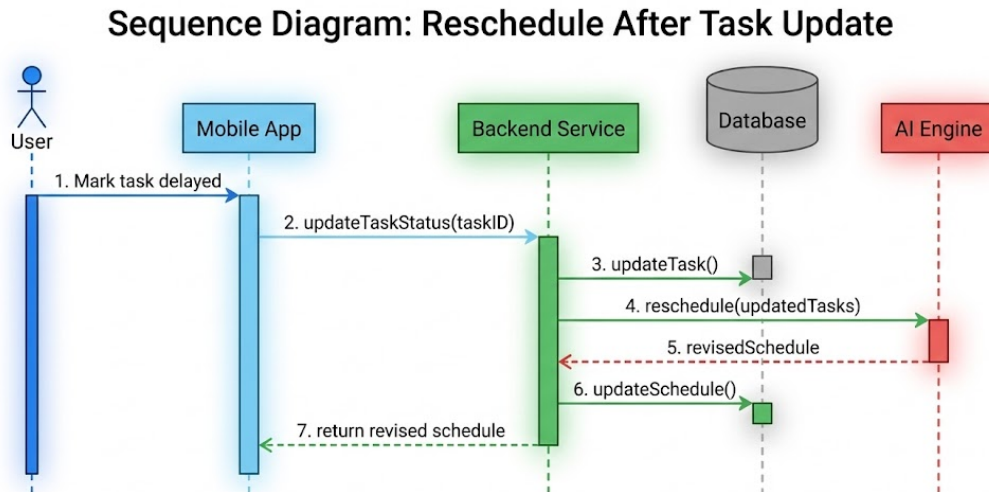


Figure 20: Sequence Diagram – Reschedule After Task Update

4.3.4 Activity Diagram – Daily Task Lifecycle

The activity diagram illustrates the complete lifecycle of a task from creation to completion. It includes task validation, scheduling, execution, and decision points for rescheduling in case of delays. The diagram provides a high-level view of control flow and decision-making involved in daily task management.

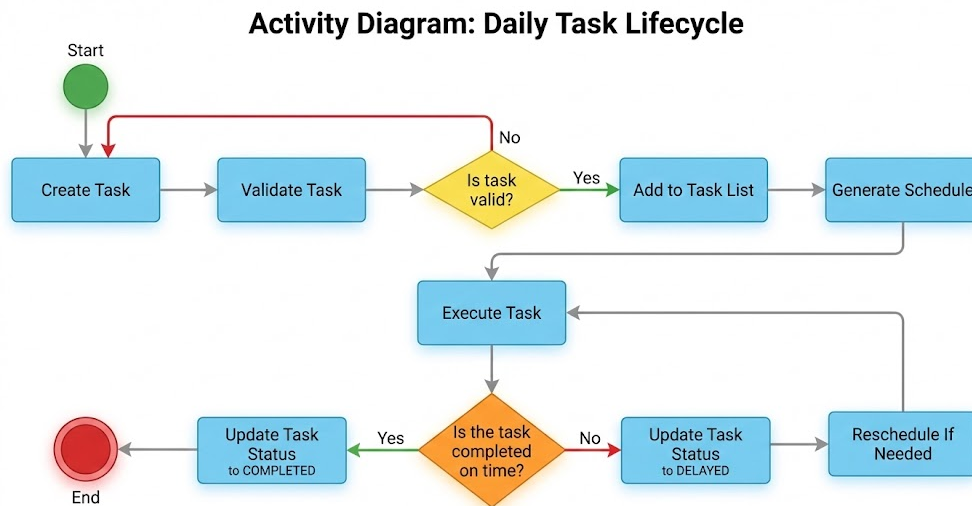


Figure 21: Activity Diagram – Daily Task Lifecycle

4.3.5 State Diagram – Task State Model

This state diagram represents the various states a task can transition through during its lifecycle. States such as Created, Scheduled, In Progress, Completed, Delayed, and Cancelled are

shown along with valid transitions. The diagram ensures consistency in task state handling and supports reliable task status management.

State Diagram: Task State Model

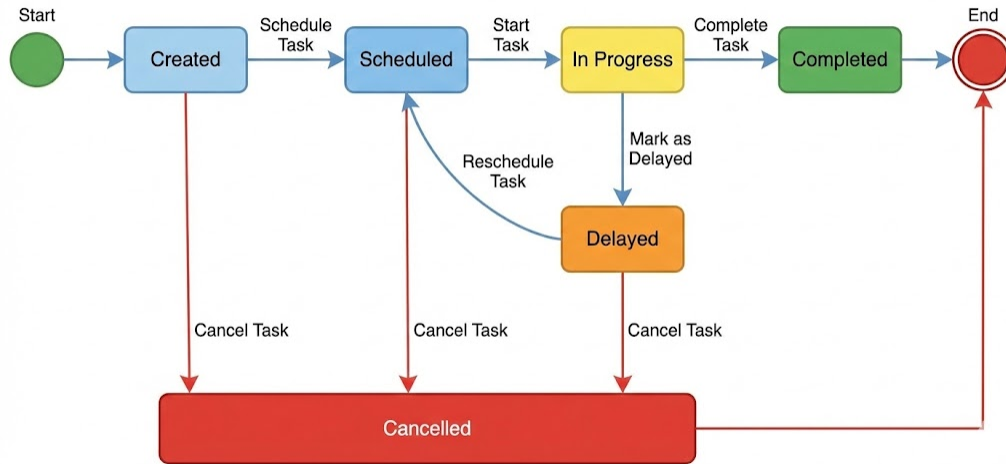


Figure 22: State Diagram – Task State Model

4.3.6 Component Diagram – Intelligent Schedule Manager

The component diagram depicts the high-level software components of the system and their interdependencies. It shows how the mobile frontend interacts with the backend API, which in turn communicates with the AI scheduling engine, database, external calendar API, and notification service. This diagram highlights modularity and separation of concerns in the system architecture.

Component Diagram: Intelligent Schedule Manager

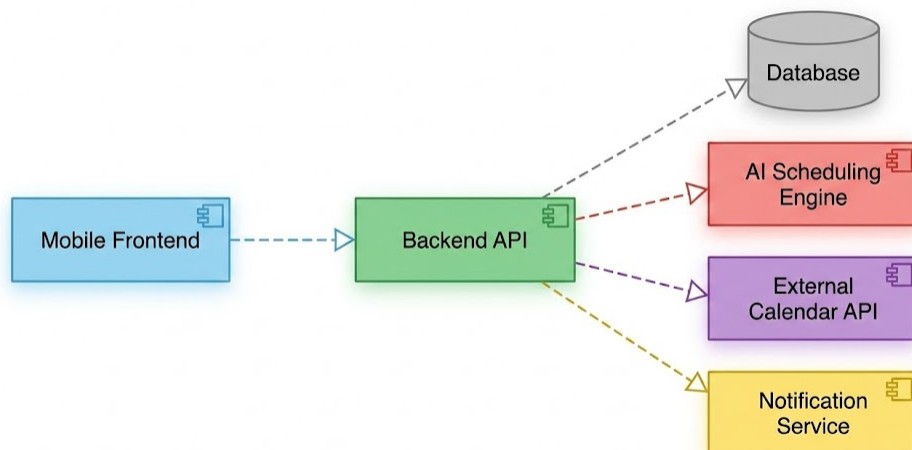


Figure 23: Component Diagram – Intelligent Schedule Manager

4.3.7 Deployment Diagram – Intelligent Schedule Manager System

The deployment diagram illustrates the physical distribution of system components across different hardware nodes. It shows communication between the user device, application server, AI processing server, database server, and external API servers using secure protocols. This diagram provides insight into scalability, security, and real-world deployment considerations.

Deployment Diagram: Intelligent Schedule Manager System

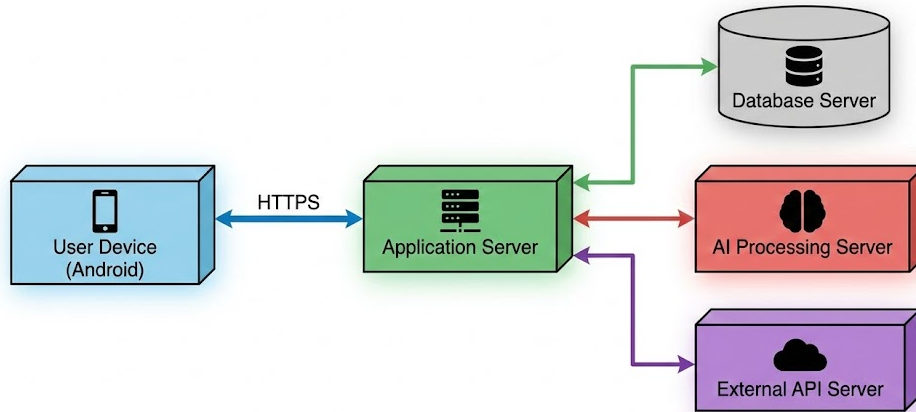


Figure 24: Deployment Diagram – Intelligent Schedule Manager System

4.4 Database Schema

The database schema follows a relational model consisting of Users, Tasks, Habits, and Schedules tables. Primary and foreign keys ensure referential integrity. Each task and habit is linked to a specific user through foreign key relationships.

The Intelligent Schedule Manager uses a relational database model to ensure structured data storage, integrity, and scalability. The schema consists of the following primary tables:

Users

- user_id (Primary Key)
- name
- email (Unique)
- password_hash
- timezone
- created_at

Preferences

- preference_id (Primary Key)
- user_id (Foreign Key)
- work_start_time
- work_end_time
- break_duration
- focus_level

Tasks

- task_id (Primary Key)
- user_id (Foreign Key)
- title
- description
- priority
- deadline
- estimated_duration
- status
- created_at

Habits

- habit_id (Primary Key)
- user_id (Foreign Key)
- name
- frequency
- streak
- last_completed_date

Schedules

- schedule_id (Primary Key)
- user_id (Foreign Key)

- schedule_date
- total_tasks

Schedule Items

- item_id (Primary Key)
- schedule_id (Foreign Key)
- task_id (Foreign Key)
- start_time
- end_time

Notifications

- notification_id (Primary Key)
- user_id (Foreign Key)
- message
- trigger_time
- delivered

Foreign key constraints enforce referential integrity between related tables. This schema supports user personalization, schedule optimization, task tracking, and secure data management.

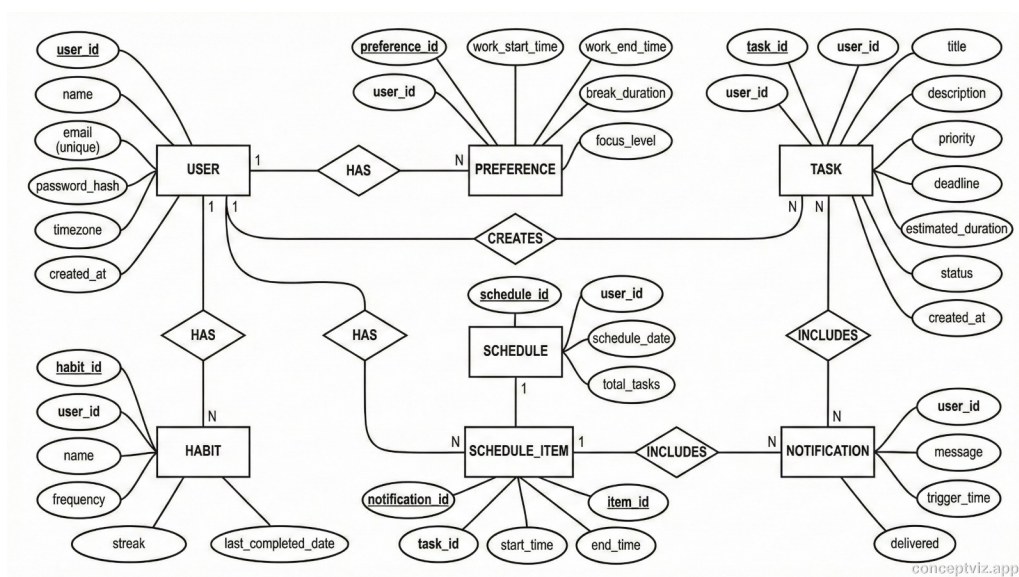


Figure 25: Entity Relationship Diagram for Intelligent Schedule Manager

4.5 Security Design

4.5.1 Authentication and Authorization Mechanisms

The system implements secure authentication using token-based mechanisms such as JWT. Passwords are securely hashed using bcrypt. Role-based access control ensures users access only their authorized data.

4.5.2 Security Protocols

All client-server communication is secured using HTTPS and TLS encryption. The system protects against SQL injection, cross-site scripting (XSS), and session hijacking. Token expiration and secure password storage enhance system security.

5 Traceability

5.1 Requirement-to-Design Mapping

Requirement ID	Requirement Name	Design ID(s)	Design Artifact
FR-1	Secure Login	D-CL-01, D-SQ-01	Class Diagram, Sequence Diagram
FR-2	Schedule Generation	D-SQ-02	Sequence Diagram
FR-3	Rescheduling	D-SQ-03	Sequence Diagram
FR-4	Calendar Sync	D-INT-01	API Interface Design
FR-5	Habit Tracking	D-CL-04	Class Diagram
NFR-03	Secure Access	D-SEC-01	Security Design

A Technology Stack

The Intelligent Schedule Manager is developed using modern cross-platform and cloud-based technologies to ensure scalability, reliability, and performance.

Frontend

- Flutter Framework for cross-platform mobile development
- Android Studio as the primary development environment
- Dart programming language

Backend

- Firebase Authentication for secure login and user management
- Node.js for backend services and API handling
- RESTful APIs for communication between frontend and backend

Database

- Firebase Firestore for cloud-based storage
- PostgreSQL for relational data management (optional deployment)

AI Engine

- Python-based scheduling algorithms
- Machine Learning models for schedule optimization

External Services

- Google Calendar API for synchronization
- Push notification services for reminders

B Development Tools

The following tools were used during design and development:

- Android Studio
- Visual Studio Code
- Flutter SDK
- Firebase Console

- Git and GitHub for version control
- LaTeX (Overleaf) for documentation
- UML diagram tools (Draw.io, Lucidchart, or StarUML)

C Database Notes

The database schema is designed to ensure data consistency and efficient querying. Primary keys uniquely identify records, while foreign keys maintain relationships between entities such as users, tasks, and schedules.

Indexes may be implemented on frequently queried fields such as:

- user_id
- schedule_date
- deadline
- notification trigger time

This ensures faster retrieval and efficient schedule generation.

D Future Enhancements

The following improvements are planned for future versions of the Intelligent Schedule Manager:

- Desktop application support (Windows, macOS, Linux)
- Web-based interface
- Collaborative scheduling for teams
- AI-based productivity analytics
- Voice assistant integration
- Offline mode with automatic synchronization
- Smart recommendations based on behavioral patterns
- Integration with wearable devices